

Meet Me In The Middle

A Full-Stack CRUD Web Application

Date: 3/22/2026

Department: Computer Science

Advisor: Julie Henderson

By Helmut Cespedes

Problem Statement

Coordinating group meetups is difficult when each person is spread across different locations. The main problems include people not wanting to travel and never being able to decide on a spot. This app aims to aid in the whole process by calculating a fair midpoint and provided venues based on everyone's preferences.

Purpose

To showcase Full-Stack developer skills with in demand architecture, tech stack, and skills.

1. Tech Stack

2. DevOps and Deployment

3. Scalability

4. System Design

5. System Integration

Research and Background

My research mainly focused on choosing a strong scalable tech stack, following common architectural patterns, pre-made functionalities, and deployment workflow.

Other research included finding the formula to calculate a midpoint based on lat/long coordinates, due to the earth's curvature its not as simply as adding and dividing by two.

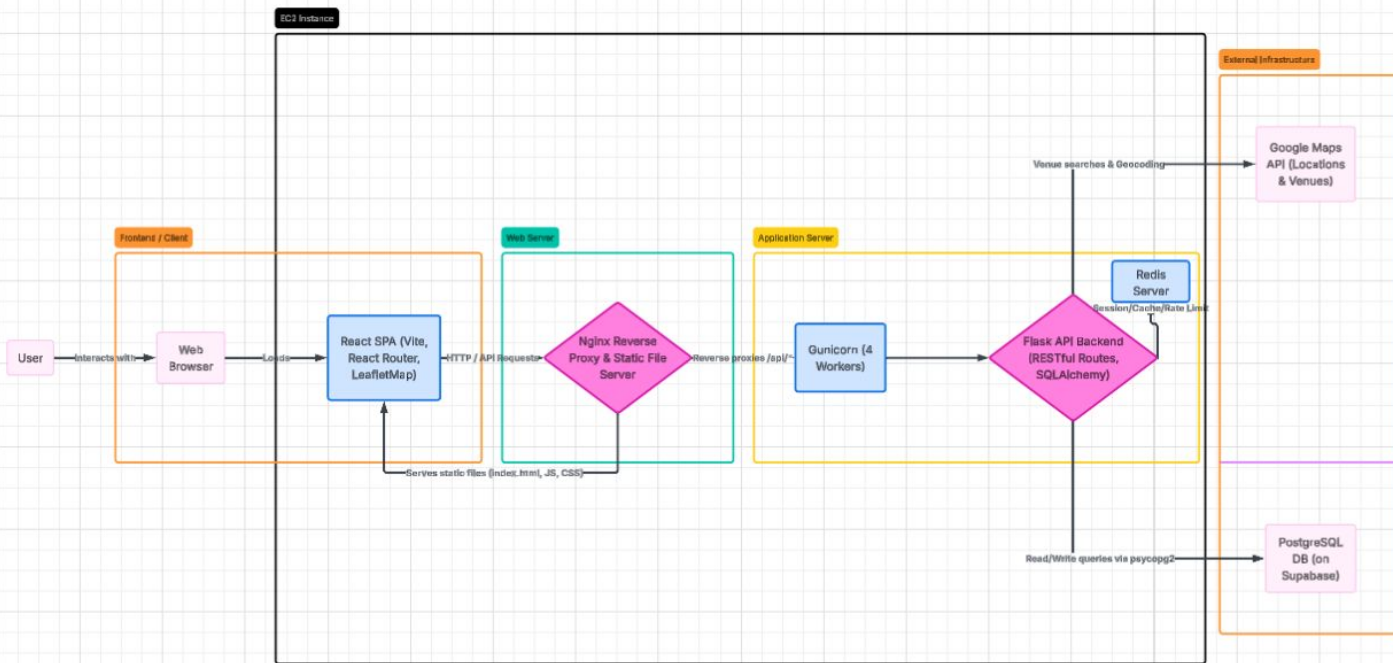
Tech Stack

Frontend	Backend	DevOps
Framework: React - React DOM and Routing - React Leaflet - Lucide React - React Hot Toast	Framework: Flask - SQLAlchemy(ORM) - Login - CORS - Limiter - Redis - Bcrypt - Psyocopg2 - Google Maps - Geopy	- Docker - Nginx(Alpine) - Gunicorn - Vite - EC2 - Supabase

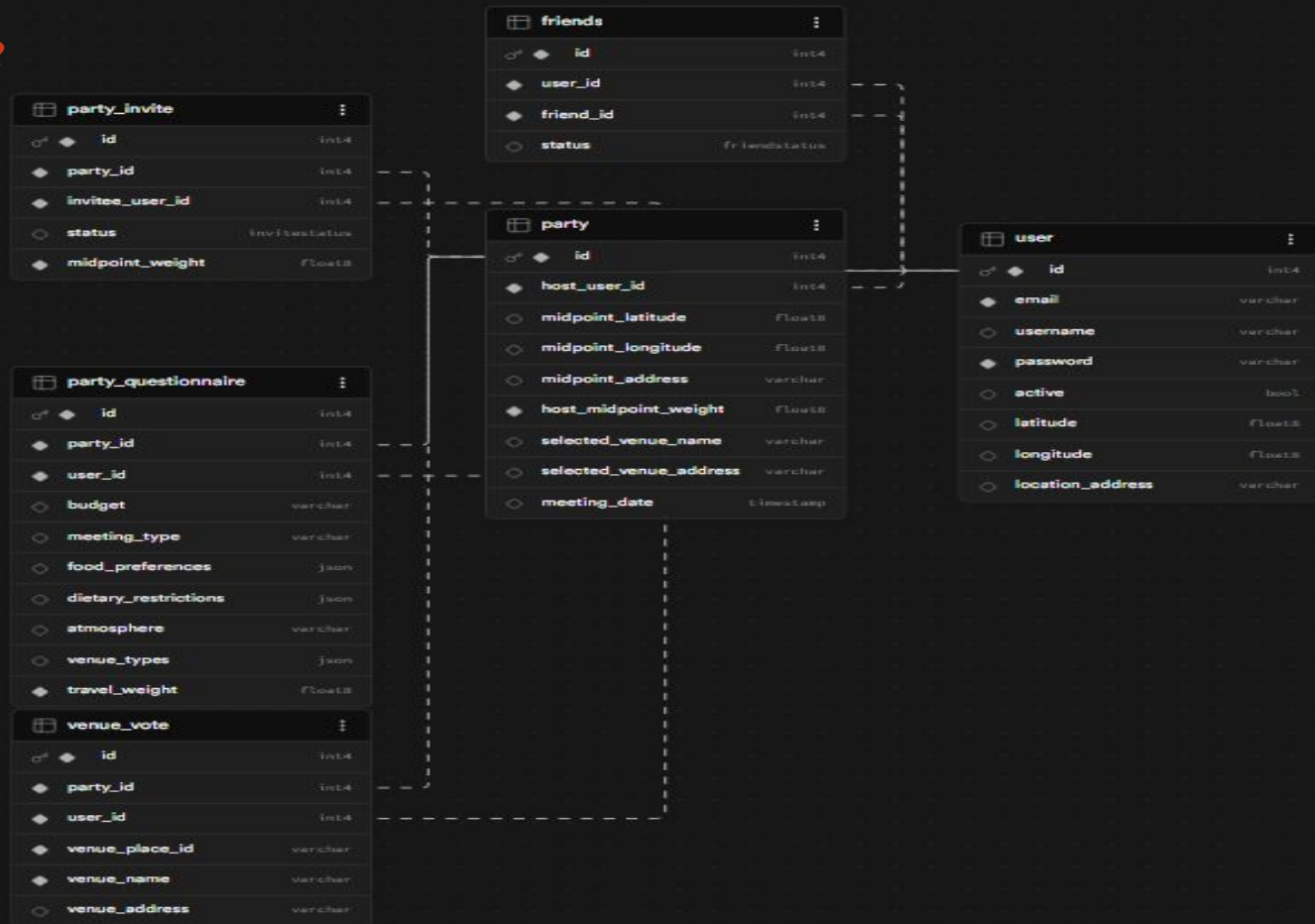
Languages:

- Python
- JS(JSX)
- SQL (ORM queries)
- CSS

Architecture



Database



Database Continued

User: stores email, username, hashed password, lat/lon, location_address.

Friends: M:M join table tracking user_id, friend_id, and

FriendStatus: (pending, accepted, declined)

Party: central meetup object tied to host_user_id. Stores midpoint coordinates, midpoint_address, meeting_date, and selected venue info.

PartyInvite: tracks invitation status (pending/accepted/declined) and individual midpoint_weight per user.

PartyQuestionnaire: stores each member's preferences, budget, meeting_type, food_preferences, dietary_restrictions, atmosphere, venue_types, and travel_weight.

VenueVote: records user votes for specific venue_place_ids linked to a Party.

Database Continued

In order to provide an application that provides a bit of scalability, I implemented a N + 1 query prevention layer. These are optimized queries that are the core component of the application.

These were implemented with SQLAlchemy eager loading (joinedload/subqueryload) so all relating data can be fetched in one query instead of having to make a call for every data piece.

All relationships use FK, constraints, and common query indexing.

Only like 3 queries were optimized the rest arent too bad of dry calls.

Midpoint Algorithm

To account for Earth's curvature, the midpoint calculation uses a 3D Cartesian coordinate transformation:

- 1.) Convert each participants lat/long into X, Y, Z space.
- 2.) Apply each user's travel_weight value from their questionnaire.
- 3.) Compute Weighted average across all the X, Y, Z vectors.
- 4.) Convert Cartesian back to lat/long.

This is kind of unnecessary since most of the U.S would be covered by a simply lat + lon/2 but was added to keep accuracy.

Route: /api/auth

Endpoints:

/register [POST] - creates new user account. Validated Input, hashes password (bcrypt), auto /login. **Rate Limit: 5/hr**

/login [POST] - authenticates user with email and password, init session, **Rate Limit: 5/min**

/logout [POST]- end user session.

/me [GET] - session management, returns current logged in user, id, email, username.

Route: /api/location + /api/venues

/save [POST] - saves the current users lat/lon to their profile for midpoint calculation.

/search [POST] - Google Maps API call, returns top 10. Cached 15 minutes. **Rate Limit: 30/min**

Route: /api/friends

Endpoints:

/add [POST] - Sends friend request by email query (Checks self-add and duplicates)

/accept [POST] - Accept pending friend request

/decline [POST]- decline pending friend request

/ [GET] - returns all accepted friend for the current user (no duplicates)

/request [GET] - returns all pending friend requests (incoming)

Route: /api/parties

Endpoints:

/create [POST] - create new party, verifies users are friends, sends invites.

/invites [GET] - returns all pending party invites for current users, includes host and invitee details.

/active [GET]- Returns user's current active party

/:id [GET] - Returns full party details (host, invitees, midpoint, finalized venue, google calendar link.

/leave [POST] - Leaves party, if host leaves, cascade delete entire party.

Route: /api/parties

Endpoints:

`/:id/midpoint [GET]` - calculates the weighted geographic midpoint from all members locations, saves to the party.

`/:id/weight [PUT]` - Update users midpoint weight

`/:id/weight [GET]`- Returns user's current midpoint weight and role.

`/:id/vote [POST]` - update venue votes

`/:id/finalize [POST](Host Only)` - picks top voted venue and sets meeting date, generates template for Google Calendar via Link.

Route: /api/parties/:id/questionnaire

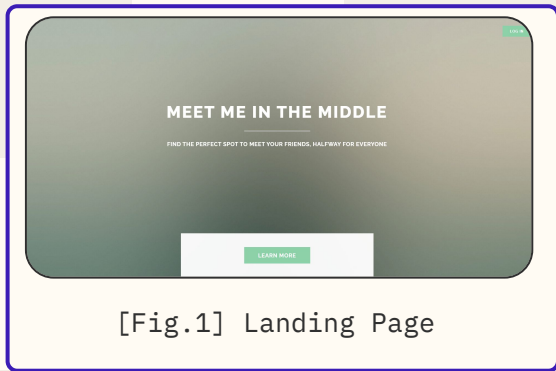
Endpoints:

/ [POST] - Submit or update preferences based on google maps form.

/ [GET] - Returns current users questionnaire (submitted) for party.

/all [GET]- returns all members questionnaire for a party.

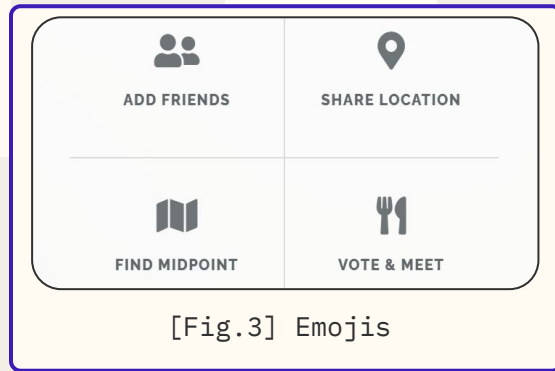
/matched-venues [GET] - combines member preferences based on algorithm.



[Fig.1] Landing Page



[Fig.2] Footer



[Fig.3] Emojis

Landing Page

Due to my time constraints i decided to build on top of a template. The template includes thousands of lines of css for every purpose I needed, basically a custom framework (eg. bootstrap, tailwind).

The template is a MIT open-source license, even so I provided the proper copyright notices at the bottom of the landing page aswell as in the codebase.

Landing Page is made up of multiple components which I cannot fit a link to: Header.jsx, Features.jsx, About.jsx, Footer.jsx, GetStarted.jsx, NavBar.jsx.

Login

For the login page, it features both client side and server side validation.

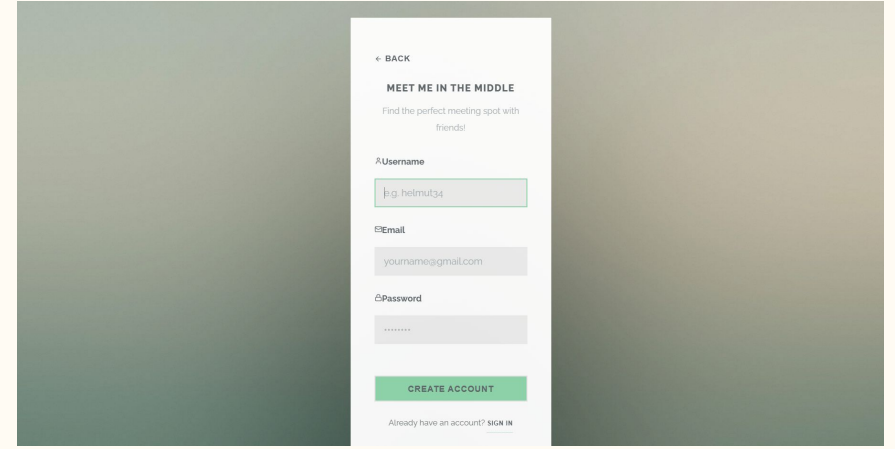
Client Side: Simple regex check and size constraints.

Backend: Received api call from auth, on success it calls the login API and sets the user "in app" state.

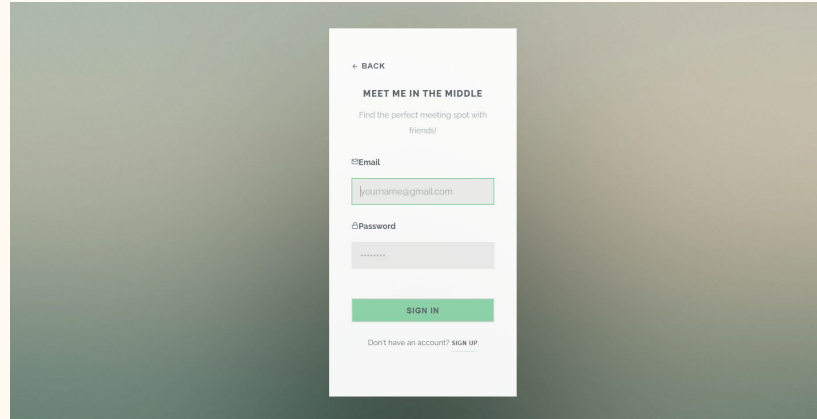
For Error handling, a backend error is displayed on based on the error type.

Optimization: Limited Bcrypt to 10 rounds.

<https://github.com/HeImut34/CSU-Senior-Project-Meet-Me/blob/master/frontend/src/components/LoginPage.jsx>

A screenshot of a web form titled "MEET ME IN THE MIDDLE" with the subtitle "Find the perfect meeting spot with friends!". The form is set against a dark green background. It includes a "+ BACK" link at the top left. The form fields are: "Username" with a placeholder "e.g. helmut34", "Email" with a placeholder "yourname@gmail.com", and "Password" with a masked input. Below the fields is a green "CREATE ACCOUNT" button. At the bottom, there is a link "Already have an account? SIGN IN".

[Fig.1] Sign Up

A screenshot of a web form titled "MEET ME IN THE MIDDLE" with the subtitle "Find the perfect meeting spot with friends!". The form is set against a dark green background. It includes a "+ BACK" link at the top left. The form fields are: "Email" with a placeholder "yourname@gmail.com" and "Password" with a masked input. Below the fields is a green "SIGN IN" button. At the bottom, there is a link "Don't have an account? SIGN UP".

[Fig.2] Sign In

Dashboard

Layout for the dashboard is simple and streamlined to avoid user confusion.

Get Location -> Add Friends -> Invite to Party

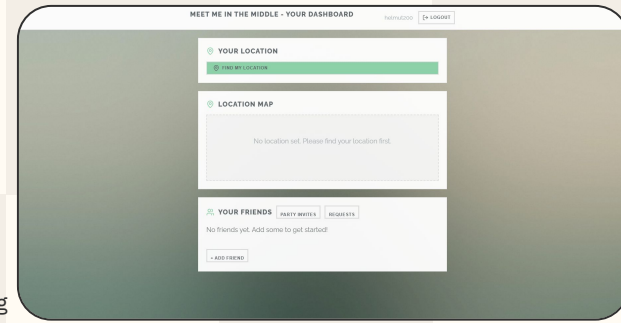
This was my heaviest page, at one point. On Load, it made 5 API roundtrips in order to display all the things on the page. To reduce wait time and make the page more responsive I got it down to 2 calls. Friend, Party, and Request are fetched on click rather than on load now.

The Location Service's speed is based on the browser, all browsers have support for this but some are slower.

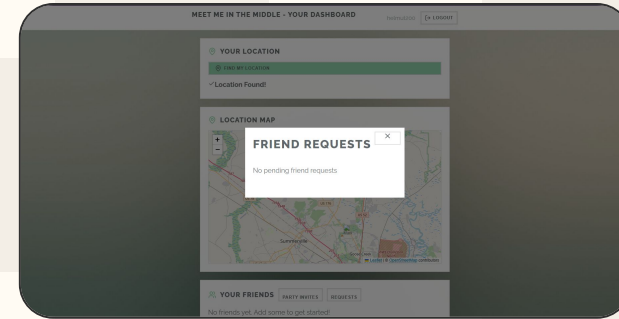
For User Feedback I used React Hot Toast. A library with prebuilt notifications for users. This is built on the regular Toast notifications. These notifications pop up on logout, send invite, accept friend, decline friend, accept party, add friend.

<https://github.com/Helmut34/CSU-Senior-Project-Meet-Me/blob/master/frontend/src/components/Dashboard.jsx>

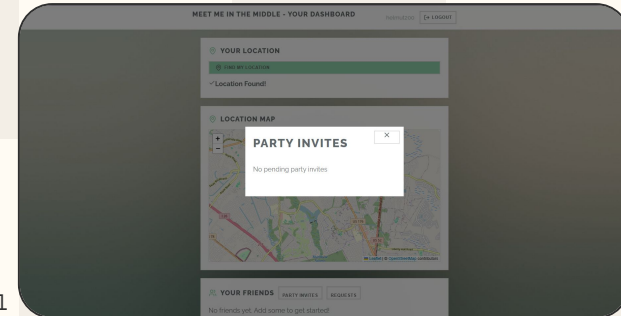
[Fig.1] Main Landing



[Fig.2] Friends Modal



[Fig.3] Party Modal



Party Page

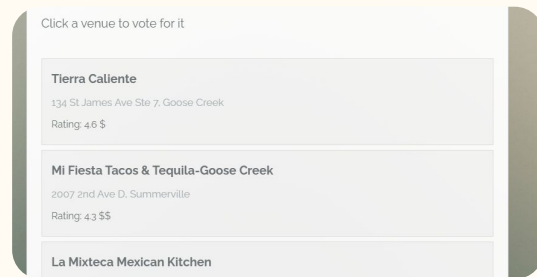
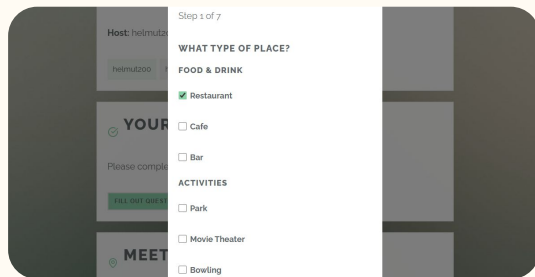
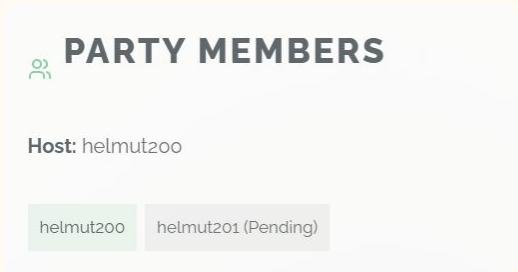
The party page features a similar design to the Dashboard to maintain fluidity.

Notice, the party questionnaire modal pops up automatically, if the user decides to click out of the modal, they may reopen via the button.

The Page shows the party state for all the different users and updates accordingly.

It also gives adequate user feedback to fill out necessary information.

Once the user has filled out the questionnaire they may calculate the midpoint which will trigger a call to the google places api, which returns an array of the 10 closest hits.



Test Plan

Test Plan Link

Test Results

This project featured unit testing for any core computational logic. The test plan provided was manually executed post-development, with all test cases passing.

User testing consisted of 5 friends and family, their feedback was given orally:

Mostly Positive, one pain point was: not clear that user must press "Find Location" button before using the rest of the functionalities.

This will be considered in future enhancements.

Challenges Overcome

- 1.) Research: Starting from such a low basepoint in knowledge led me to make mistakes that cost me in the future, while the whole workflow is very generic and so were the tutorial i followed, there are so many add-ons that everyone preferred over others that it became overwhelming to choose what to implement and what's unnecessary.
- 2.) Migrating Backend: While setting up my serving, Docker wouldn't let me use some of my imports (Flask Security) because it had a dependency that wasn't even used in their most recent version. I had to switch to a Flask Login + Bcrypt implementation (Not a huge change).

Challenges Overcome

4.) Template CSS was ridiculous, over 3k lines. I had to pick out what I wanted and didn't.

5.) Workaround serving over HTTPS instead of HTTP in order for location services to work.

6.) CORS and Session Management setup, horrible explanations from tutorial videos, basically just expect you to copy and paste.

Future Enhancements

- API were influenced by what the frontend needed, bad practice. Make them more generic.
- SEO audit
- Make Functions more less “need based” to lower coupling.
- Separate Services to different instances in order to minimize faults. Right now if one thing goes down the whole app doesn't work.